Exercise 1 - From metric to Cartesian (origin and translated centers)

Question

Starting from $\|(x,y)-(x_0,y_0)\| = R$, derive $(x-x_0)^2+(y-y_0)^2=R^2$. Then specialize to $C=(0,0)$ to recover $x^2+y^2=R^2$. Explain why squaring preserves the locus here (no extraneous branches), and note what would go wrong if the right-hand side were negative.

Start from the metric (Euclidean-distance) definition of a circle with center $C=(x_0,y_0)$ and radius $R \geq 0$:

$$\|(x,y)-(x_0,y_0)\| = R.$$

By definition of the Euclidean norm,

$$\|(x,y)-(x_0,y_0)\| = \sqrt{(x-x_0)^2+(y-y_0)^2}.$$

Thus

$$\sqrt{(x-x_0)^2+(y-y_0)^2} = R.$$

Both sides are **nonnegative**: the square root is ≥ 0 and $R \geq 0$ by convention for a radius. On the domain $[0,\infty)$, the function $t \mapsto t^2$ is strictly increasing, so squaring preserves equality and introduces **no extraneous solutions**:

$$(x-x_0)^2+(y-y_0)^2 = R^2.$$

Specializing to the origin $C=(0,0)$ (a translation by $(-x_0,-y_0)$) gives the familiar Cartesian form:

$$x^2+y^2=R^2.$$

Why squaring is safe here. The usual pitfall with squaring an equation $A=B$ is that it can add solutions whenever A and B might differ by sign (e.g., $A=-B$ also implies $A^2=B^2$). Here, however, $A=\sqrt{(x-x_0)^2+(y-y_0)^2} \geq 0$ and $B=R \geq 0$. Because both sides are constrained to be nonnegative, the only way $A^2=B^2$ can hold is $A=B$. Hence the locus is preserved exactly.

What would go wrong if the right-hand side were negative.
If someone wrote

$$(x - x_0)^2 + (y - y_0)^2 = -a^2 \quad (a > 0),$$

the left side is always ≥ 0 while the right side is < 0; there are **no** real (x, y) satisfying this —i.e., the locus is empty. Likewise, if one started with $\|(x, y) - (x_0, y_0)\| = R$ but (incorrectly) took $R < 0$, the metric equation has no solution because a distance cannot be negative; squaring would disguise this by yielding the formally valid but misleading R^2, which is nonnegative. This is another reason we fix $R \geq 0$ before squaring.

Exercise 2 - Recognition via the quadratic part

Question

Consider the general conic $ax^2 + 2bxy + cy^2 + 2dx + 2ey + f = 0$. Treat the quadratic part as the symmetric matrix $A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$. Prove that if the curve is a circle then A must be a scalar multiple of the identity (hence $b = 0$ and $a = c$). Use the coordinate rotation R_θ and the invariance condition $R_\theta^\top A R_\theta = A$ for all θ.

We start from the general second-degree (conic) equation

$$ax^2 + 2bxy + cy^2 + 2dx + 2ey + f = 0,$$

and isolate its quadratic part via the symmetric matrix

$$A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}, \qquad Q(x, y) = \begin{pmatrix} x & y \end{pmatrix} \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = ax^2 + 2bxy + cy^2.$$

If the curve is a circle, then after translating coordinates to its center C (so that the linear terms vanish), the equation takes the form

$$x^2 + y^2 = R^2 \quad (\text{in the centered coordinates}).$$

Thus, in centered coordinates the locus is invariant under every rotation R_θ. Writing this invariance for the quadratic form says exactly

$$Q(R_\theta \mathbf{x}) = Q(\mathbf{x}) \quad \text{for all } \theta \text{ and all } \mathbf{x} \in \mathbb{R}^2.$$

In matrix form this is

$$\mathbf{x}^\top R_\theta^\top A R_\theta \mathbf{x} = \mathbf{x}^\top A \mathbf{x} \quad \text{for all } \mathbf{x},$$

which is equivalent to the matrix identity

$$(*) R_\theta^\top A R_\theta \;=\; A \qquad \text{for all } \theta \in \mathbb{R}. \tag{*}$$

Now compute $R_\theta^\top A R_\theta$ explicitly and enforce $(*)$. Let

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} = \begin{pmatrix} u & -v \\ v & u \end{pmatrix}, \quad \text{where } u = \cos\theta,\ v = \sin\theta,\ u^2 + v^2 = 1.$$

First multiply AR_θ:

$$AR_\theta = \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} u & -v \\ v & u \end{pmatrix} = \begin{pmatrix} au + bv & -av + bu \\ bu + cv & -bv + cu \end{pmatrix}.$$

Then multiply on the left by $R_\theta^\top = \begin{pmatrix} u & v \\ -v & u \end{pmatrix}$:

$$R_\theta^\top (AR_\theta) = \begin{pmatrix} u & v \\ -v & u \end{pmatrix} \begin{pmatrix} au + bv & -av + bu \\ bu + cv & -bv + cu \end{pmatrix}.$$

Compute each entry:

• $(1,1)$-entry:

$$u(au + bv) + v(bu + cv) = au^2 + 2buv + cv^2.$$

• $(1,2)$-entry:

$$u(-av + bu) + v(-bv + cu) = -auv + b(u^2 - v^2) + cuv = (c - a)uv + b(u^2 - v^2).$$

• By symmetry, the $(2,1)$-entry equals the $(1,2)$-entry (since A is symmetric and rotations preserve symmetry):

$$(2,1) = (1,2) = (c - a)uv + b(u^2 - v^2).$$

• $(2,2)$-entry:

$$(-v)(-av + bu) + u(-bv + cu) = av^2 - 2buv + cu^2.$$

Thus

$$R_\theta^\top AR_\theta = \begin{pmatrix} au^2 + 2buv + cv^2 & (c - a)uv + b(u^2 - v^2) \\ (c - a)uv + b(u^2 - v^2) & av^2 - 2buv + cu^2 \end{pmatrix}.$$

Imposing $R_\theta^\top AR_\theta = A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ for all θ means the following identities must hold for all (u, v) on the unit circle:

\begin{aligned} a &= a u^2 + 2 b u v + c v^2, \tag{E1}\\ b &= (c-a)u v + b(u^2 - v^2), \tag{E2}\\ c &= a v^2 - 2 b u v + c u^2. \tag{E3} \end{aligned}

Choose two special angles to force the parameters:

1. Take $\theta = \frac{\pi}{2}$ so $u = 0,\ v = 1$. Then (E1) gives $a = c$. (E3) gives $c = a$ again (consistent).

2. With $a = c$ now known, take $\theta = \frac{\pi}{4}$ so $u = v = \frac{1}{\sqrt{2}}$. Then $u^2 - v^2 = 0$ and $uv = \frac{1}{2}$. Equation (E2) becomes

$$b = (c - a) \cdot \tfrac{1}{2} + b \cdot 0 = 0.$$

Therefore $b = 0$ and $a = c$. Hence

$$A = \begin{pmatrix} a & 0 \\ 0 & a \end{pmatrix} = a\, I_2,$$

a scalar multiple of the identity. This is exactly the isotropy of the quadratic form of a circle: the squared radius is the same in every direction.

A short alternative (same conclusion, different algebra) is to differentiate $(*)$ at $\theta = 0$. Writing $R_\theta = I + \theta J + O(\theta^2)$ with $J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, the condition $R_\theta^\top A R_\theta = A$ for all θ implies the commutator vanishes:

$$0 = \frac{d}{d\theta}\Big|_{\theta=0} (R_\theta^\top A R_\theta) = -JA + AJ \quad \Longrightarrow \quad AJ = JA.$$

Writing $A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ and J as above, $AJ = JA$ yields

$$\begin{pmatrix} b & -a \\ c & -b \end{pmatrix} = \begin{pmatrix} -b & -c \\ a & b \end{pmatrix} \implies a = c,\ b = 0.$$

Since A is symmetric, this again forces $A = aI_2$.

Either way, if the (centered) conic is a circle, the quadratic part is a scalar multiple of the identity, i.e. $b = 0$ and $a = c$.

```python
# Retry with a simpler (and more robust) symmetry test:
# Enforce R^T A R = A at three distinct angles and solve for (a,b,c).
# Also include the infinitesimal-rotation commutator proof.

import sympy as sp

a, b, c = sp.symbols('a b c', real=True)
theta = sp.symbols('theta', real=True)

R = sp.Matrix([[sp.cos(theta), -sp.sin(theta)],
               [sp.sin(theta),  sp.cos(theta)]])
A = sp.Matrix([[a, b],
               [b, c]])

RtAR = sp.simplify(R.T * A * R)
Delta = sp.simplify(RtAR - A)

# Build equations by sampling Δ(θ) at a small set of angles (enough to kill sin/cos components)
angles = [0, sp.pi/4, sp.pi/2]
eqs = []
for ang in angles:
    Dv = sp.simplify(Delta.subs(theta, ang))
    # require every entry to be zero at this angle
    for i in range(2):
        for j in range(2):
            eqs.append(sp.Eq(sp.simplify(Dv[i,j]), 0))

sol = sp.solve(eqs, (a,b,c), dict=True)

print("=== Solving invariance from R^T A R = A at θ ∈ {0, π/4, π/2} ===")
print("Equations:")
for e in eqs:
    print(" ", e)
print("\nSolutions for (a,b,c):", sol)
```

```
35
36  # Derive explicit relations rather than a concrete triple.
37  # Typically SymPy gives a relation set; if it returns an empty
    list (meaning underdetermined),
38  # we can compute relations from a single symbolic off-diagonal
    entry.
39  if not sol:
40      off = sp.simplify(Delta[0,1])
41      off2 = sp.simplify(sp.expand_trig(off.rewrite(sp.sin)))
42      print("\nOff-diagonal Δ12(θ):", off2)
43      # Express off2 = A*sin(2θ) + B*cos(2θ). Extract A and B by
    evaluating at θ=0 and θ=π/4.
44      Acoef = sp.simplify(sp.diff(off2,
    sp.sin(2*theta)).subs({theta: sp.pi/4}))
45      Bcoef = sp.simplify(sp.diff(off2,
    sp.cos(2*theta)).subs({theta: 0}))
46      print("Heuristic coefficients A (sin 2θ) and B (cos 2θ):",
    Acoef, Bcoef)
47
48  # Infinitesimal method
49  J = sp.Matrix([[0, -1],
50                 [1,  0]])
51  comm = sp.simplify(A*J - J*A)
52  eqs_comm = [sp.Eq(comm[0,0],0), sp.Eq(comm[0,1],0),
    sp.Eq(comm[1,0],0), sp.Eq(comm[1,1],0)]
53  sol_comm = sp.solve(eqs_comm, (a,b,c), dict=True)
54
55  print("\n=== Infinitesimal method ([A,J]=0) ===")
56  print("Commutator [A,J]:")
57  sp.pprint(comm)
58  print("\nSolutions for (a,b,c) from [A,J]=0:", sol_comm)
59
60  # As a final sanity check, verify that A = α I satisfies
    invariance symbolically.
61  alpha = sp.symbols('alpha', real=True)
62  A_iso = alpha * sp.eye(2)
63  check = sp.simplify(R.T * A_iso * R - A_iso)
64  print("\nSanity check: with A = α I, R^T A R - A equals")
```

```
sp.pprint(check)

```

Exercise 3 - Sufficiency of isotropy and completing the square

Question

Assume $b = 0$ and $a = c \neq 0$. Divide the equation by a and rename parameters to obtain $x^2 + y^2 + 2mx + 2ny + q = 0$. Complete the square in x and y to produce $(x + m)^2 + (y + n)^2 = m^2 + n^2 - q$. Identify the center and radius, and specify precisely the three regimes $m^2 + n^2 - q > 0, = 0$, and < 0.

Start from the isotropic quadratic (after you've assumed $b = 0$ and $a = c \neq 0$):

$$ax^2 + ay^2 + 2dx + 2ey + f = 0.$$

1. **Normalize and rename parameters**

Divide by $a \neq 0$:

$$x^2 + y^2 + \frac{2d}{a}x + \frac{2e}{a}y + \frac{f}{a} = 0.$$

Introduce

$$m := \frac{d}{a}, \qquad n := \frac{e}{a}, \qquad q := \frac{f}{a},$$

so the equation is

$$(1)x^2 + y^2 + 2mx + 2ny + q = 0. \tag{1}$$

2. **Complete the square in x and y**

Group x-terms and y-terms:

$$(x^2 + 2mx) + (y^2 + 2ny) + q = 0.$$

Add and subtract the needed squares, m^2 and n^2, inside the corresponding groups:

$$\left(x^2 + 2mx + m^2\right) - m^2 \; + \; \left(y^2 + 2ny + n^2\right) - n^2 \; + \; q = 0.$$

Rewrite each completed square:

$$(x + m)^2 + (y + n)^2 - (m^2 + n^2) + q = 0.$$

Move the constants to the right:

$$(2)(x+m)^2+(y+n)^2=m^2+n^2-q. \tag{2}$$

3. **Identify center and radius, with regimes**

Equation (2) is a circle in standard form provided the right-hand side is nonnegative.

- **Center**: $C=(-m,\,-n)$.
- **Squared radius**: $R^2=m^2+n^2-q$.
- **Radius** (when defined): $R=\sqrt{m^2+n^2-q}$.

Now distinguish the three precise cases:

1. $m^2+n^2-q>0$:
   A genuine circle of radius $R=\sqrt{m^2+n^2-q}$ centered at $(-m,-n)$.
2. $m^2+n^2-q=0$:
   A **point circle** (degenerate): the locus reduces to the single point $(-m,-n)$.
3. $m^2+n^2-q<0$:
   No real points: the left-hand side $(x+m)^2+(y+n)^2\geq 0$ cannot equal a negative number, so the locus is empty.
4. **(Check) Expand to verify**

From (2),

$$(x+m)^2+(y+n)^2=x^2+2mx+m^2+y^2+2ny+n^2=x^2+y^2+2mx+2ny+(m^2+n^2),$$

so setting this equal to m^2+n^2-q yields exactly $x^2+y^2+2mx+2ny+q=0$, which matches (1).

```
# SymPy check: normalize, complete the square, extract center & radius, and classify regimes symbolically.

import sympy as sp

# Symbols
x, y = sp.symbols('x y', real=True)
a, d, e, f = sp.symbols('a d e f', real=True, nonzero=True)

# Assume isotropy: b=0, a=c != 0; start from a x^2 + a y^2 + 2 d x + 2 e y + f = 0
expr = a*x**2 + a*y**2 + 2*d*x + 2*e*y + f

# Normalize by a and rename
m, n, q = sp.symbols('m n q', real=True)
subs_map = {m: d/a, n: e/a, q: f/a}
expr_norm = sp.simplify((expr / a).xreplace({d/a: m, e/a: n, f/a: q}))  # xreplace to keep m,n,q explicitly

# Complete the square: (x+m)^2 + (y+n)^2 - (m^2 + n^2 - q) == 0
completed = sp.expand((x + m)**2 + (y + n)**2 - (m**2 + n**2 - q))

# Extract center and radius^2
center = (-m, -n)
R2 = sp.simplify(m**2 + n**2 - q)

print("Normalized equation:")
sp.pprint(sp.Eq(expr_norm, 0))

print("\nCompleted-square form:")
sp.pprint(sp.Eq(completed, 0))

print("\nCenter C and squared radius R^2:")
print("C =", center)
print("R^2 =", R2)

```

```
# Regime classification (symbolic description)
print("\nRegimes:")
print("1) R^2 > 0  → circle of radius sqrt(R^2).")
print("2) R^2 = 0  → point circle at C.")
print("3) R^2 < 0  → empty locus (no real points).")

```

Exercise 4 - Translation as a linear-algebra step

Question

Rewrite the implicit equation in matrix form $X^\top AX + 2\ell^\top X + f = 0$ with $X = (x, y)^\top, A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$, and $\ell = (d, e)^\top$. Show that translating $X = Y + h$ removes the linear terms exactly when $Ah = -\ell$. Solve for h when $A = \lambda I$ and interpret $h = (-m, -n)$ from the completed-square form as the circle's center.

Write the conic in quadratic-form notation

$$X^\top AX + 2\,\ell^\top X + f = 0, \qquad X = \begin{pmatrix} x \\ y \end{pmatrix},\ A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}\ (\text{symmetric}),\ \ell = \begin{pmatrix} d \\ e \end{pmatrix}.$$

We translate by a vector $h \in \mathbb{R}^2$ via $X = Y + h$. Substitute and expand carefully:

$$0 = Y\top AY \ + \ Y\top Ah \ + \ h\top AY \ + \ h\top Ah \ + \ 2\,\ell\top Y \ + \ 2\,\ell\top h \ + \ f. \qquad \begin{aligned} 0 &= (Y+h)^\top A(Y+h) + 2\,\ell^\top(\\ &= Y^\top AY \ + \ Y^\top Ah \ + \ h^\top A\end{aligned}$$

Since A is symmetric, $Y^\top Ah = h^\top AY$. Group by powers of Y:

$$0 = Y^\top AY \ + \ 2\,(Ah + \ell)^\top Y \ + \ \underbrace{\left(h^\top Ah + 2\,\ell^\top h + f\right)}_{\text{new constant}}.$$

When do the linear terms vanish?

Exactly when the coefficient of Y is zero:

$$Ah + \ell = 0 \quad \Longleftrightarrow \quad Ah = -\,\ell.$$

This is both necessary and sufficient. In that case the translated equation becomes

$$Y^\top AY + f' = 0, \qquad f' := h^\top Ah + 2\,\ell^\top h + f,$$

i.e., no linear terms remain.

Solve $Ah = -\ell$ when $A = \lambda I$

If $A = \lambda I$ with $\lambda \neq 0$ (the isotropic case),

$$\lambda I\, h = -\,\ell \implies h = -\frac{1}{\lambda}\,\ell = -\frac{1}{\lambda}\begin{pmatrix} d \\ e \end{pmatrix} = \begin{pmatrix} -d/\lambda \\ -e/\lambda \end{pmatrix}.$$

Match to the completed-square parameters

From Exercise 3, after dividing the original equation by a (here λ) we wrote

$$x^2 + y^2 + 2mx + 2ny + q = 0, \qquad m = \frac{d}{a},\ n = \frac{e}{a}.$$

Completing the square gave

$$(x+m)^2 + (y+n)^2 = m^2 + n^2 - q,$$

so the center is $(-m, -n) = \big(-d/a, -e/a\big)$.

This is exactly the translation vector h we just computed:

$$h = \begin{pmatrix} -d/\lambda \\ -e/\lambda \end{pmatrix} \quad \text{with } \lambda = a \implies h = (-m, -n),$$

i.e., the linear-algebra "solve $Ah = -\ell$" is the same operation as "complete the square," and h is the circle's center. (If $\lambda = 0$ or A is singular/non-isotropic, you either have no circle or need a different analysis.)

```
# SymPy program for the translation X = Y + h that removes linear terms in X^T A X + 2 ℓ^T X + f = 0.
# - Shows that the linear term vanishes iff A h = -ℓ
# - Solves h symbolically (when det(A) ≠ 0)
# - Specializes to the isotropic case A = λ I to get h = -(1/λ) ℓ
# - Connects to completed-square parameters for a circle: (b=0, a=c=λ) ⇒ h = (-m, -n) with m=d/λ, n=e/λ
# - Provides a helper to compute the translated constant term f' = h^T A h + 2 ℓ^T h + f
# - Includes a quick numeric demo
#
# Plain printed output.

import sympy as sp

# Symbols
a, b, c, d, e, f = sp.symbols('a b c d e f', real=True)
x, y = sp.symbols('x y', real=True)

# Build A, ℓ, X
A = sp.Matrix([[a, b],
               [b, c]])
ell = sp.Matrix([d, e])
X = sp.Matrix([x, y])

# Generic translation vector h
h1, h2 = sp.symbols('h1 h2', real=True)
h = sp.Matrix([h1, h2])
Y = X - h  # (we'll also use X = Y + h equivalently)

# Original quadratic form and translated form
F = (X.T * A * X)[0] + 2*(ell.T * X)[0] + f
F_translated = ((Y + h).T * A * (Y + h))[0] + 2*(ell.T * (Y + h))[0] + f
F_translated_simplified = sp.expand(F_translated)

```

```
# Extract the linear-in-Y coefficient vector by taking gradient wrt Y and evaluating at general Y
Y1, Y2 = sp.symbols('Y1 Y2', real=True)
Ysym = sp.Matrix([Y1, Y2])
F_Y = ((Ysym + h).T * A * (Ysym + h))[0] + 2*(ell.T * (Ysym + h))[0] + f
gradY = sp.Matrix([sp.diff(F_Y, Y1), sp.diff(F_Y, Y2)])
linear_coeff = sp.simplify(gradY.subs({Y1:0, Y2:0}))  # coefficient of Y (since gradient at 0)

print("F(X) = X^T A X + 2 ℓ^T X + f")
print("A ="); sp.pprint(A)
print("ℓ ="); sp.pprint(ell)
print("\nTranslated form F(Y+h) expanded:")
sp.pprint(F_translated_simplified)

print("\nLinear-in-Y coefficient (should be 2*(A h + ℓ)):")
sp.pprint(linear_coeff)

print("\nCondition to remove linear terms: A h + ℓ = 0  ⇔  A h = -ℓ")
Ah_plus_ell = sp.simplify(A*h + ell)
print("A h + ℓ =")
sp.pprint(Ah_plus_ell)

# Solve A h = -ℓ symbolically
print("\nSolve A h = -ℓ (when det(A) ≠ 0):")
detA = sp.simplify(A.det())
print("det(A) =", detA)
h_solution = sp.simplify(A.LUsolve(-ell))  # assumes det(A) != 0
print("h* ="); sp.pprint(h_solution)

# Translated constant term f' = h^T A h + 2 ℓ^T h + f
fprime = sp.simplify((h_solution.T * A * h_solution)[0] + 2*(ell.T * h_solution)[0] + f)
print("\nf' (translated constant) = h^T A h + 2 ℓ^T h + f evaluated at h*")
sp.pprint(fprime)
```

```
65
66  # Specialization: isotropic case A = λ I
67  lam = sp.symbols('lam', real=True, nonzero=True)
68  A_iso = lam * sp.eye(2)
69  h_iso = sp.simplify(A_iso.LUsolve(-ell))
70  print("\nIsotropic case A = λ I ⇒ h = -(1/λ) ℓ:")
71  print("h_iso ="); sp.pprint(h_iso)
72
73  # Connection to completed square for the circle case (b=0,
    a=c=λ):
74  # In normalized circle form x^2 + y^2 + 2 m x + 2 n y + q = 0, we
    have m = d/λ, n = e/λ → center = (-m, -n)
75  m_sym, n_sym, q_sym = sp.symbols('m_sym n_sym q_sym', real=True)
76  m_from_coeffs = sp.simplify(d/lam)
77  n_from_coeffs = sp.simplify(e/lam)
78  center_from_completed_square = sp.Matrix([-m_from_coeffs, -
    n_from_coeffs])
79  print("\nCompleted-square center for circle with a=c=λ, b=0:  (-
    m, -n) = (-d/λ, -e/λ)")
80  sp.pprint(center_from_completed_square)
81  print("Matches h_iso:", sp.simplify(center_from_completed_square
    - h_iso) == sp.Matrix([0,0]))
82
83  # Quick numeric demo
84  print("\nNumeric demo: a=c=4, b=0, d=6, e=-8, f=9  ⇒  x^T A x + 2
    ℓ^T x + f = 0")
85  demo_subs = {a:4, b:0, c:4, d:6, e:-8, f:9}
86  h_num = sp.N(h_solution.subs(demo_subs))
87  detA_num = sp.N(detA.subs(demo_subs))
88  fprime_num = sp.N(fprime.subs(demo_subs))
89  print("det(A) =", detA_num)
90  print("h* =", h_num, "  (this should equal the circle center)")
91  print("f' =", fprime_num, "  (constant term after translation)")
92
```

Exercise 5 - Necessity via curvature isotropy (geometric argument)

Question

Give a coordinate-free proof that a circle's quadratic part must be isotropic: argue that because the circle looks the same in every direction, the principal curvatures of the quadratic form at the center must agree. Translate this geometric statement into $a = c$ and $b = 0$ without using rotation matrices, then reconcile your argument with Exercise 2.

Start by translating to the circle's center so the linear terms disappear. Write the (centered) implicit equation as

$$F(X) = X^\top A X + \text{const} = 0, \qquad X = (x, y)^\top, \quad A = A^\top.$$

The "quadratic part" at the center is the symmetric bilinear form $B(U, V) = U^\top A V$ with associated quadratic form $Q(U) = B(U, U) = U^\top A U$. Geometrically, if you intersect the curve with any line through the center in direction of a unit vector u, you get—up to translation—a 1-dimensional quadratic $g(t) = Q(tu) + \text{const} = t^2\, Q(u) + \text{const}$. The curvature of that 1-D intersection at the center depends on the second derivative $g''(0) = 2\, Q(u)$. A true circle "looks the same in every direction" at its center, so this directional second derivative (hence the normal curvature there) must be the same for every unit direction u. In symbols:

$$(*) \quad \boxed{Q(u) = u^\top A u \ \text{ is constant for all unit } u \in \mathbb{S}^1.} \tag{*}$$

A symmetric matrix A whose quadratic form is constant on the unit circle must be a scalar multiple of the identity. To see this without invoking rotations, fix an orthonormal basis e_1, e_2 and set

$$\alpha := e_1^\top A e_1, \quad \gamma := e_2^\top A e_2, \quad \beta := e_1^\top A e_2 = e_2^\top A e_1.$$

For a unit vector $u = (\cos\theta)\, e_1 + (\sin\theta)\, e_2$ we have

$$Q(u) = u^\top A u = \alpha \cos^2\theta + 2\beta \sin\theta \cos\theta + \gamma \sin^2\theta.$$

If $Q(u)$ is independent of θ, its derivative with respect to θ must vanish for all θ:

$$\frac{d}{d\theta} Q(u) = (\gamma - \alpha)\sin\theta\cos\theta + 2\beta(\cos^2\theta - \sin^2\theta) \equiv 0 \ \text{ for all } \theta.$$

The only way a trigonometric polynomial of the form
$(\gamma - \alpha)\sin\theta\cos\theta + 2\beta(\cos^2\theta - \sin^2\theta)$
vanishes for every θ is if both coefficients are zero:

$$\gamma - \alpha = 0 \quad \text{and} \quad \beta = 0.$$

Hence $\alpha = \gamma$ and $\beta = 0$. In matrix entries,

$$A = \begin{pmatrix} \alpha & \beta \\ \beta & \gamma \end{pmatrix} = \begin{pmatrix} \alpha & 0 \\ 0 & \alpha \end{pmatrix} = \alpha\, I_2.$$

Returning to the original notation $A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$, this is exactly

$$a = c \quad \text{and} \quad b = 0.$$

This proves—purely from "same in every direction" at the center (curvature isotropy)—that the quadratic part must be isotropic.

To reconcile with Exercise 2: there we formalized "looks the same in every direction" by requiring full rotational invariance $R_\theta^\top A R_\theta = A$ for all θ; differentiating at $\theta = 0$ or expanding for specific angles again forces A to commute with all rotations, hence $A = \lambda I$. Here, instead of the matrix identity, we equated directional second derivatives (equivalently, the restriction of Q to $\mathbb{S}^1$) and obtained the same conclusion. Both arguments encode the same geometry: isotropy at the center implies A is a scalar multiple of the identity, i.e. $a = c$ and $b = 0$.

The following snippet differentiates $Q(u) = u^\top A u$ w.r.t. θ and solves the "independent of θ" condition:

```python
import sympy as sp

a,b,c,theta = sp.symbols('a b c theta', real=True)
u = sp.Matrix([sp.cos(theta), sp.sin(theta)])
A = sp.Matrix([[a,b],[b,c]])

Q = (u.T * A * u)[0]                    # u^T A u
dQ = sp.simplify(sp.diff(Q, theta))   # must vanish for all theta
# Enforce coefficient-wise zero by evaluating at enough angles:
sol = sp.solve([
    sp.Eq(dQ.subs(theta, 0), 0),
    sp.Eq(dQ.subs(theta, sp.pi/4), 0),
    sp.Eq(dQ.subs(theta, sp.pi/2), 0),
], (a,b,c), dict=True)
# Returns [{a: c, b: 0}]

```

Exercise 6 - Degeneracies: point or imaginary locus

Question

Begin with $x^2 + y^2 + 2mx + 2ny + q = 0$. Show carefully that $m^2 + n^2 - q = 0$ yields the single point $(x, y) = (-m, -n)$ and that $m^2 + n^2 - q < 0$ yields no real solutions. As a check, construct an explicit numerical example for each case and verify by inspection.

Start from

$$(1)x^2 + y^2 + 2mx + 2ny + q = 0. \tag{1}$$

Complete the square in x and y **algebraically and explicitly**:

$$= (x+m)2 - m2, y2 + 2ny = (y+n)2 - n2, \quad \begin{matrix} x^2 + 2mx = (x+m)^2 - m^2, \\ y^2 + 2ny = (y+n)^2 - n^2, \end{matrix}$$

so adding these to q gives

$$\left[(x+m)^2 - m^2\right] + \left[(y+n)^2 - n^2\right] + q = 0.$$

Collect constants on the right:

$$(2)(x+m)^2 + (y+n)^2 = m^2 + n^2 - q. \tag{2}$$

Set

$$S := m^2 + n^2 - q.$$

Case 1: $S = 0$ ***(point circle)***

Equation (2) becomes

$$(x+m)^2 + (y+n)^2 = 0.$$

Each square is **nonnegative** over R, and a sum of nonnegatives equals 0 **iff each term is 0**. Therefore

$$(x+m)^2 = 0 \;\Rightarrow\; x = -m, \qquad (y+n)^2 = 0 \;\Rightarrow\; y = -n.$$

Hence the unique solution is the single point $(x, y) = (-m, -n)$.

Check by back-substitution. Plug $(x, y) = (-m, -n)$ into (1):

$$= m2 + n2 + 2m(-m) + 2n(-n) + q = m2 + n2 - 2m2 - 2n2 + q = -(m2 + n2) + q = -(m2 + n2) +$$

as required.

Concrete example. Take $m = 1,\ n = 2,\ q = 5$. Then $S = 1^2 + 2^2 - 5 = 0$ and the only solution is $(-1, -2)$. Inspect (1):

$$x^2 + y^2 + 2x + 4y + 5 = 0.$$

At $(-1, -2)$: $1 + 4 - 2 - 8 + 5 = 0$. Any other point makes the left side positive (since it equals $(x + 1)^2 + (y + 2)^2$).

Case 2: $S < 0$ (no real locus)

Equation (2) asks for a sum of two real squares to equal a **negative** number:

$$(x + m)^2 + (y + n)^2 = S < 0.$$

But $(x + m)^2 \geq 0$ and $(y + n)^2 \geq 0$ for all real x, y, hence their sum is ≥ 0. Therefore no real (x, y) can satisfy (2). Equivalently, the locus is empty (an "imaginary circle" in classical language).

Concrete example. Take $m = 1,\ n = 2,\ q = 6$. Then $S = 1^2 + 2^2 - 6 = -1 < 0$. The equation is

$$x^2 + y^2 + 2x + 4y + 6 = 0,$$

which rewrites as $(x + 1)^2 + (y + 2)^2 = -1$, impossible over R. By inspection, even at the "best" point $(-1, -2)$ (the center candidate), the left side is

$$1 + 4 - 2 - 8 + 6 = 1 > 0,$$

and moving away only increases $(x + 1)^2 + (y + 2)^2$, so there are no solutions.

```python
# SymPy verification for Exercise 6: degeneracies of x^2 + y^2 + 2mx + 2ny + q = 0
import sympy as sp

# Symbols
x, y, m, n, q = sp.symbols('x y m n q', real=True)

# Original and completed-square forms
expr = x**2 + y**2 + 2*m*x + 2*n*y + q
S = sp.simplify(m**2 + n**2 - q)
completed = sp.expand((x + m)**2 + (y + n)**2 - S)   # equals original LHS

print("Original equation:  x^2 + y^2 + 2mx + 2ny + q = 0")
print("Completed square:   (x + m)^2 + (y + n)^2 = S,  where S := m^2 + n^2 - q\n")
print("SymPy check that LHS equals completed-square form minus S:", sp.simplify(expr - ((x+m)**2 + (y+n)**2 - S)) == 0)

# Case 1: S = 0  → unique point (-m, -n)
sol_point = sp.solve([sp.Eq((x+m)**2 + (y+n)**2, 0)], (x, y), dict=True)
print("\nCase S = 0: Solve (x + m)^2 + (y + n)^2 = 0")
print("Solutions:", sol_point, "  (over the reals, squares sum to 0 iff each is 0).")

# Verify back-substitution gives 0 in the original
x0, y0 = -m, -n
check_point = sp.simplify(expr.subs({x: x0, y: y0, q: m**2 + n**2}))
print("Back-substitution at (-m, -n) with q = m^2 + n^2 →", check_point)

# Case 2: S < 0  → no real solutions
# Analytical certificate: min_{x,y} (x+m)^2 + (y+n)^2 = 0, achieved at (-m,-n); cannot equal negative S.
# We'll illustrate with a concrete example.
m_val, n_val = 1, 2
```

```
30
31  # Example: S = 0 (point circle)
32  q_point = m_val**2 + n_val**2
33  expr_point = expr.subs({m: m_val, n: n_val, q: q_point})
34  lhs_at_center_point = sp.simplify(expr_point.subs({x: -m_val, y:
    -n_val}))
35
36  # Example: S < 0 (no real locus)
37  q_imag = m_val**2 + n_val**2 + 1   # so S = -1
38  expr_imag = expr.subs({m: m_val, n: n_val, q: q_imag})
39  lhs_at_center_imag = sp.simplify(expr_imag.subs({x: -m_val, y: -
    n_val}))
40
41  print("\nConcrete examples with m=1, n=2:")
42  print("  Example S=0:   q =", q_point, " → equation: x^2 + y^2 +
    2x + 4y + 5 = 0")
43  print("    LHS at (-1, -2):", lhs_at_center_point, " (should be
    0)")
44
45  print("  Example S<0:  q =", q_imag, " → equation: x^2 + y^2 + 2x
    + 4y + 6 = 0")
46  print("    LHS at (-1, -2):", lhs_at_center_imag, " (minimal
    possible; still > 0 ⇒ no real solutions)")
47
48  # Bonus: demonstrate the minimum of (x+m)^2 + (y+n)^2 over reals
    is 0
49  min_val = sp.simplify(sp.minimum((x+m)**2 + (y+n)**2, x),
    rational=True) if hasattr(sp, "minimum") else 0
50  print("\nAnalytic fact: min_{x,y∈R} (x+m)^2 + (y+n)^2 = 0 at
    (x,y)=(-m,-n).")
51
```

Exercise 7 - Intersection with a line and the discriminant test

Question

Intersect the general circle $(x - x_0)^2 + (y - y_0)^2 = R^2$ with the line $y = px + r$. Reduce to a quadratic in x and compute its discriminant. Show that tangency occurs exactly

when the discriminant vanishes, and rewrite that condition as the distance-from-center-to-line equaling R.

We'll intersect the circle

$$(x - x_0)^2 + (y - y_0)^2 = R^2$$

with the line

$$y = px + r \qquad (\text{non-vertical}; p \in \mathbb{R}).$$

Substitute $y = px + r$ into the circle and expand completely:

$$0 = (x2 - 2x0x + x02) + (p2x2 + 2px(r - y0) + (r - y0)2) - R2 = (1 + p2)x2 + (-2x0 + 2p(r - y0))x$$

Thus we get a quadratic in x:

$$Ax^2 + Bx + C = 0,$$

with

$$A = 1 + p^2, \qquad B = 2\big(-x_0 + p(r - y_0)\big), \qquad C = x_0^2 + (r - y_0)^2 - R^2.$$

Compute the discriminant $\Delta = B^2 - 4AC$. It is convenient to set $s := r - y_0$ for the algebra and then substitute back:

$$\Delta 4 = (x02 - 2px0s + p2s2) - ((1 + p2)x02 + (1 + p2)s2 - (1 + p2)R2) = -(p2x02 + 2px0s + s2) + (1$$

Undo $s = r - y_0$:

$$px_0 + s = px_0 + (r - y_0) = px_0 - y_0 + r.$$

Therefore

$$\boxed{\Delta = 4\Big((1 + p^2)R^2 - (px_0 - y_0 + r)^2\Big)}.$$

Tangency condition and distance reinterpretation

Tangency occurs exactly when the quadratic has a **double root**, i.e. when $\Delta = 0$. From the boxed formula,

$$\Delta = 0 \quad \Longleftrightarrow \quad (1+p^2)R^2 = (px_0 - y_0 + r)^2.$$

Take square roots (both sides nonnegative):

$$\frac{|px_0 - y_0 + r|}{\sqrt{1+p^2}} = R.$$

But

$$\operatorname{dist}\big((x_0, y_0),\ y = px + r\big) = \frac{|px_0 - y_0 + r|}{\sqrt{1+p^2}},$$

so the tangency criterion is exactly

$$\boxed{\text{line is tangent} \iff \operatorname{dist}((x_0, y_0),\ y = px + r) = R}.$$

(As a corollary: $\Delta > 0$ gives two intersection points; $\Delta < 0$ gives no real intersection.)

Remark on vertical lines. If the line is $x = x_*$ (vertical), substitute $x = x_*$ instead and solve the quadratic in y; you obtain the analogous tangency condition $|x_* - x_0| = R$, which is the same distance-equals-radius statement.

```python
# SymPy verification for Exercise 7: circle ∩ line, discriminant, and tangency = distance condition
import sympy as sp

# Symbols
x, y = sp.symbols('x y', real=True)
x0, y0, R = sp.symbols('x0 y0 R', real=True)
p, r = sp.symbols('p r', real=True)

# Circle and line
circle = (x - x0)**2 + (y - y0)**2 - R**2
line = sp.Eq(y, p*x + r)

# Substitute y from the line into the circle to get a quadratic in x
expr_x = sp.expand(circle.subs(y, p*x + r))

# Identify quadratic coefficients A x^2 + B x + C
A = sp.expand(sp.diff(expr_x, x, 2) / 2)
B = sp.expand(sp.diff(expr_x, x) - 2*A*x)  # remove the 2Ax part from the first derivative
C = sp.expand(expr_x.subs({x: 0}))

Delta = sp.expand(B**2 - 4*A*C)

# Expected discriminant form: 4*((1+p**2)*R**2 - (p*x0 - y0 + r)**2)
Delta_expected = sp.expand(4*((1+p**2)*R**2 - (p*x0 - y0 + r)**2))

print("Quadratic in x: A x^2 + B x + C = 0")
print("A =", A)
print("B =", B)
print("C =", C)

print("\nDiscriminant Δ = B^2 - 4 A C")
print("Δ  =", Delta)
```

```
print("Δ_expected =", Delta_expected)
print("Matches:", sp.simplify(Delta - Delta_expected) == 0)

# Tangency condition: Δ = 0  ⇔  |p x0 - y0 + r| / sqrt(1+p^2) = R
distance = sp.Abs(p*x0 - y0 + r) / sp.sqrt(1 + p**2)

print("\nTangency (Δ=0)  ⇔  (1+p^2) R^2 = (p x0 - y0 + r)^2  ⇔  distance = R")
print("distance(center, line) =", distance)

```

Exercise 8 - Removing the mixed term by rotation

Question

For $ax^2 + 2bxy + cy^2 = 1$ with $b \neq 0$, perform a rotation $x = u\cos\theta - v\sin\theta, y = u\sin\theta + v\cos\theta$ and choose θ so that the uv term vanishes. Derive $\tan 2\theta = \frac{2b}{a-c}$. Conclude that even after eliminating the mixed term, unequal diagonal coefficients ($a \neq c$) survive unless $a = c$, so the curve cannot be a circle.

We start from the quadratic form

$$ax^2 + 2bxy + cy^2 = 1, \qquad b \neq 0,$$

and rotate coordinates by an angle θ:

$$x = u\cos\theta - v\sin\theta, \qquad y = u\sin\theta + v\cos\theta.$$

For brevity write $c_\theta := \cos\theta$ and $s_\theta := \sin\theta$.

1. Expand $x^2,\ xy,\ y^2$ in (u, v)

$$x2y2 = (us\theta + vc\theta)2 = u2s\theta2 + 2uvs\theta c\theta + v2c\theta2, xy = (uc\theta - vs\theta)(us\theta + vc\theta) = u2c\theta s\theta + uv(c\theta2 - s\theta2$$

2. Collect coefficients of $u^2,\ uv,\ v^2$

Insert into $ax^2 + 2bxy + cy^2$:

- Coefficient of u^2:

$$ac_\theta^2 + 2b(c_\theta s_\theta) + cs_\theta^2.$$

- Coefficient of uv:

$$a(-2c_\theta s_\theta) + 2b(c_\theta^2 - s_\theta^2) + c(2s_\theta c_\theta).$$

Group the $c_\theta s_\theta$ terms:

$$(-2a + 2c)c_\theta s_\theta + 2b(c_\theta^2 - s_\theta^2) = 2(c - a)c_\theta s_\theta + 2b(c_\theta^2 - s_\theta^2).$$

- Coefficient of v^2:

$$as_\theta^2 - 2b(s_\theta c_\theta) + cc_\theta^2.$$

Hence the rotated equation is

$$\left[\alpha(\theta)\right] u^2 + \left[\beta(\theta)\right] uv + \left[\gamma(\theta)\right] v^2 = 1,$$

with

$$\alpha(\theta) = ac_\theta^2 + \\ = ac\theta 2 + 2bc\theta s\theta + cs\theta 2, \beta(\theta) = 2(c - a)c\theta s\theta + 2b(c\theta 2 - s\theta 2), \gamma(\theta) = as\theta 2 - 2bs\theta c\theta + cc\theta 2. \beta(\theta) = 2(c - \\ \gamma(\theta) = as_\theta^2 -$$

3. Choose θ to remove the mixed term

We want $\beta(\theta) = 0$. Use the double-angle identities

$$\sin 2\theta = 2c_\theta s_\theta, \qquad \cos 2\theta = c_\theta^2 - s_\theta^2.$$

Then

$$\beta(\theta) = (c - a)\sin 2\theta + 2b\cos 2\theta.$$

Set $\beta(\theta) = 0$:

$$(c - a)\sin 2\theta + 2b\cos 2\theta = 0.$$

If $\cos 2\theta \neq 0$, divide through:

$$(c - a)\tan 2\theta + 2b = 0 \quad \Longrightarrow \quad \tan 2\theta = \frac{2b}{a - c}.$$

This is the standard diagonalizing angle. (If $\cos 2\theta = 0$, then $\tan 2\theta$ is infinite and the same formula still holds in the limiting sense when $a = c$ with $b \neq 0$; see below.)

4. Unequal diagonal coefficients persist unless $a = c$

With this θ, the mixed term vanishes and we obtain a diagonal form

$$\lambda_1 u^2 + \lambda_2 v^2 = 1,$$

where $\lambda_{1,2}$ are the (real) eigenvalues of the symmetric matrix $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$:

$$\lambda_{1,2} = \frac{a+c}{2} \pm \frac{1}{2}\sqrt{(a-c)^2 + 4b^2}.$$

Unless $b = 0$ **and** $a = c$, these eigenvalues are unequal ($\lambda_1 \neq \lambda_2$), so after rotation the level set is an axis-aligned ellipse or hyperbola, not a circle. In particular, even when the mixed term is removed ($\beta = 0$), unequal diagonal coefficients remain if $a \neq c$; hence the quadratic form is anisotropic, and the curve cannot be a circle.

5. Special cases (consistency checks)

- If $a = c$ and $b \neq 0$: then $\tan 2\theta = \infty \Rightarrow 2\theta = \frac{\pi}{2} \pmod{\pi}$, i.e. $\theta = \frac{\pi}{4} \pmod{\frac{\pi}{2}}$. The mixed term is removed by a 45° rotation, but the resulting coefficients are $a \pm b$, unequal because $b \neq 0$. Not a circle.
- If $b = 0$ and $a \neq c$: the mixed term is already absent, but coefficients differ, so again not a circle.
- Only if $b = 0$ **and** $a = c$ (i.e., the quadratic part is a scalar multiple of the identity) do we get $\lambda_1 = \lambda_2 = a$, which—after appropriate translation of the linear terms—produces a genuine circle.

Thus, removing the uv term by rotation (via $\tan 2\theta = \dfrac{2b}{a-c}$) does not by itself make the curve a circle; isotropy ($a = c$ and $b = 0$) is required.

```
# SymPy program for Exercise 8: rotate to remove the mixed term and report the diagonalized coefficients.
# - Uses the identity tan(2θ) = 2b / (a - c), implemented via θ = 0.5 * atan2(2b, a - c).
# - Confirms the mixed term vanishes at θ*, and returns the diagonal entries (which equal the eigenvalues).
# - Provides a convenience function with optional numeric substitution.
#
# No plots, just symbolic computation.

import sympy as sp

# Symbols
a, b, c = sp.symbols('a b c', real=True)
u, v = sp.symbols('u v', real=True)
theta = sp.symbols('theta', real=True)

# Rotation matrix R(θ) taking (u,v) -> (x,y)
R = sp.Matrix([[sp.cos(theta), -sp.sin(theta)],
               [sp.sin(theta),  sp.cos(theta)]])

# Quadratic-form matrix for ax^2 + 2bxy + cy^2
A = sp.Matrix([[a, b],
               [b, c]])

# Rotated quadratic form in (u,v): [u v] * (R^T A R) * [u v]^T
B = sp.simplify(R.T * A * R)

# Extract coefficients so that ax^2+2bxy+cy^2 becomes α u^2 + 2 β u v + γ v^2
alpha_theta = sp.simplify(B[0,0])
beta_theta  = sp.simplify(B[0,1])  # this is "β"; mixed term is 2*β u v
gamma_theta = sp.simplify(B[1,1])

# Closed-form rotation that kills the mixed term: tan(2θ) = 2b/(a-c)  ⇒ θ* = 1/2 atan2(2b, a-c)
```

```python
theta_star = sp.simplify(sp.atan2(2*b, a - c) / 2)

# Evaluate coefficients at θ*
alpha_star = sp.simplify(alpha_theta.subs(theta, theta_star))
beta_star  = sp.simplify(beta_theta.subs(theta, theta_star))
gamma_star = sp.simplify(gamma_theta.subs(theta, theta_star))

# Eigenvalues of A for reference (should match alpha_star and gamma_star up to ordering)
eigvals = [sp.simplify(ev) for ev in A.eigenvals().keys()]

# Package in a function
def diagonalize_quadratic(a_sym=a, b_sym=b, c_sym=c, subs=None):
    """
    Return a dict with: theta_star, alpha_star, beta_star (should be 0), gamma_star,
    and eigenvalues of A. If 'subs' is provided (dict mapping symbols to numbers),
    evaluate numerically as well.
    """
    res = {
        "theta_star": theta_star,
        "alpha_theta": alpha_theta,
        "beta_theta": beta_theta,
        "gamma_theta": gamma_theta,
        "alpha_star": alpha_star,
        "beta_star": beta_star,
        "gamma_star": gamma_star,
        "eigenvalues": eigvals,
    }
    if subs:
        res_num = {
            "theta_star_num": sp.N(theta_star.subs(subs)),
            "alpha_star_num": sp.N(alpha_star.subs(subs)),
            "beta_star_num": sp.N(beta_star.subs(subs)),
            "gamma_star_num": sp.N(gamma_star.subs(subs)),
```

```
            "eigenvalues_num": [sp.N(ev.subs(subs)) for ev in eigvals],
        }
        res.update(res_num)
    return res

# Demo 1 (symbolic): show the mixed term vanishes
print("β(θ*) =", beta_star)  # should simplify to 0

# Demo 2 (numeric): pick a,b,c and show diagonalized coefficients and angle
example = {a: 5, b: 3, c: 1}
res = diagonalize_quadratic(subs=example)

print("\nExample with a=5, b=3, c=1")
for k in ["theta_star_num", "alpha_star_num", "beta_star_num", "gamma_star_num", "eigenvalues_num"]:
    print(f"{k}: {res[k]}")

# Show that alpha_star/gamma_star match eigenvalues (order may differ)
print("\nSymbolic eigenvalues of A:")
for ev in eigvals:
    print("  ", ev)

print("\nDiagonalized quadratic at θ*:  alpha*u^2 + gamma*v^2 = 1 (mixed term removed)")

```

Exercise 9 - Three-point determination and non-collinearity

Question

Given three non-collinear points $P_i = (x_i, y_i)$, set up the linear system in m, n, q for $x_i^2 + y_i^2 + 2mx_i + 2ny_i + q = 0$ (all i). Show that the determinant of the 3×3 system is twice the signed area of the triangle (P_1, P_2, P_3); deduce uniqueness of the circle through the three points and explain why collinearity makes the system singular.

Start from the isotropic Cartesian form

$$x^2 + y^2 + 2mx + 2ny + q = 0.$$

Impose it at three given points $P_i = (x_i, y_i)$ (assumed non-collinear). For each $i = 1, 2, 3$,

$$x_i^2 + y_i^2 + 2m\,x_i + 2n\,y_i + q = 0 \iff 2x_i\,m + 2y_i\,n + 1 \cdot q = -(x_i^2 + y_i^2).$$

Collecting the three equations gives the 3×3 linear system in (m, n, q):

$$(1)\quad \underbrace{\begin{pmatrix} 2x_1 & 2y_1 & 1 \\ 2x_2 & 2y_2 & 1 \\ 2x_3 & 2y_3 & 1 \end{pmatrix}}_{=:C} \begin{pmatrix} m \\ n \\ q \end{pmatrix} = - \begin{pmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ x_3^2 + y_3^2 \end{pmatrix}. \tag{1}$$

Determinant and (signed) area

Introduce the "geometry" matrix with columns $(x, y, 1)$:

$$M := \begin{pmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{pmatrix}.$$

The (signed) area A_{sgn} of the oriented triangle (P_1, P_2, P_3) is

$$A_{\text{sgn}} = \frac{1}{2}\,\det M,$$

so $\det M = 2A_{\text{sgn}}$. Now relate C and M: C is obtained from M by multiplying the first two columns by 2. Determinants scale multiplicatively with column scalings, so

$$(2)\quad \det C = 2 \cdot 2 \cdot \det M = 4\,\det M = 4\,(2A_{\text{sgn}}) = 8\,A_{\text{sgn}}. \tag{2}$$

Thus $\det C$ is proportional to the signed area, vanishing **iff** the three points are collinear. (If you instead set up the system in the unknowns $(2m, 2n, q)$ with coefficient matrix M, then its determinant is $\det M = 2A_{\text{sgn}}$; equation (2) simply reflects the factor-of-4 rescaling of the first two columns.)

Uniqueness and the collinearity obstruction

- If the points are **non-collinear**, then $A_{\text{sgn}} \neq 0$, hence $\det C \neq 0$ by (2). The system (1) has a unique solution (m, n, q). That unique solution determines a unique circle in the class $x^2 + y^2 + 2mx + 2ny + q = 0$. (Equivalently, there is a unique center $(-m, -n)$ and radius $R^2 = m^2 + n^2 - q$.)
- If the points are **collinear**, then $A_{\text{sgn}} = 0$, hence $\det C = 0$. The system is singular: either there is no solution (most lines do not lie on any circle of this form) or there are infinitely many "solutions" corresponding to the geometric fact that no single circle can pass through three collinear points unless it degenerates. In our normalized form

that degeneracy would force $R^2 \le 0$ or collapse to a point—impossible for three distinct collinear points—so generically there is **no** solution.

(Optional) Cramer's-rule expression (to see the geometry explicitly)

If desired, the center $(x_0, y_0) = (-m, -n)$ can be written by Cramer's rule using M. Define

$$D = \det\begin{pmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{pmatrix} = 2A_{\text{sgn}}, \quad D_x = \det\begin{pmatrix} -(x_1^2+y_1^2) & y_1 & 1 \\ -(x_2^2+y_2^2) & y_2 & 1 \\ -(x_3^2+y_3^2) & y_3 & 1 \end{pmatrix}, \quad D_y = \det\begin{pmatrix} x_1 & -(x_1^2+y_1^2) & 1 \\ x_2 & -(x_2^2+y_2^2) & 1 \\ x_3 & -(x_3^2+y_3^2) & 1 \end{pmatrix}.$$

Solving the M-version of (1) for $(2m, 2n, q)$ gives

$$m = -\frac{D_x}{D}, \qquad n = -\frac{D_y}{D}, \qquad (x_0, y_0) = \left(\frac{D_x}{D}, \frac{D_y}{D}\right),$$

which is well defined exactly when $D \neq 0$ (i.e., the points are not collinear).

```python
# Exercise 9 – SymPy program (plain output, no special formatting)
# - Builds the 3x3 linear system for (m, n, q)
# - Proves det(C) = 8 * A_signed symbolically
# - Solves for (m, n, q), center, and R^2 for non-collinear examples
# - Detects/flags the collinear (singular) case
#
# Keep it simple: just print/pprint the results.

import sympy as sp

# Symbols
x1, y1, x2, y2, x3, y3 = sp.symbols('x1 y1 x2 y2 x3 y3', real=True)
m, n, q = sp.symbols('m n q', real=True)

# Matrices
C = sp.Matrix([
    [2*x1, 2*y1, 1],
    [2*x2, 2*y2, 1],
    [2*x3, 2*y3, 1],
])
rhs = -sp.Matrix([x1**2 + y1**2, x2**2 + y2**2, x3**2 + y3**2])

M = sp.Matrix([
    [x1, y1, 1],
    [x2, y2, 1],
    [x3, y3, 1],
])

# Determinant/area relation
detC = sp.simplify(C.det())
detM = sp.simplify(M.det())
A_signed = sp.simplify(detM / 2)  # oriented (signed) area

print("Linear system:  C * [m, n, q]^T = rhs")
```

```
sp.pprint(C)
sp.pprint(rhs)

print("\nDeterminant/area relation:")
print("det(C) =", detC)
print("det(M) =", detM)
print("A_signed = det(M)/2 =", A_signed)
print("Check det(C) - 8*A_signed =", sp.simplify(detC - 8*A_signed))

# Solver function
def solve_three_point_circle(P1, P2, P3, tol=sp.Float("1e-12")):
    """Given three points Pk=(xk,yk), return dict with solution or collinearity flag."""
    Xi = [sp.nsimplify(P1[0]), sp.nsimplify(P2[0]), sp.nsimplify(P3[0])]
    Yi = [sp.nsimplify(P1[1]), sp.nsimplify(P2[1]), sp.nsimplify(P3[1])]
    Cnum = sp.Matrix([
        [2*Xi[0], 2*Yi[0], 1],
        [2*Xi[1], 2*Yi[1], 1],
        [2*Xi[2], 2*Yi[2], 1],
    ])
    rhsnum = -sp.Matrix([Xi[0]**2 + Yi[0]**2, Xi[1]**2 + Yi[1]**2, Xi[2]**2 + Yi[2]**2])
    Mnum = sp.Matrix([
        [Xi[0], Yi[0], 1],
        [Xi[1], Yi[1], 1],
        [Xi[2], Yi[2], 1],
    ])
    dM = sp.N(Mnum.det())
    if abs(dM) < tol:
        return {"collinear": True, "detM": dM}
    sol = Cnum.LUsolve(rhsnum)
    mval, nval, qval = map(sp.simplify, sol)
    center = sp.Matrix([-mval, -nval])
    R2 = sp.simplify(mval**2 + nval**2 - qval)
```

```
    return {
        "collinear": False,
        "m": sp.N(mval), "n": sp.N(nval), "q": sp.N(qval),
        "center": sp.N(center),
        "R2": sp.N(R2),
        "detM": sp.N(dM),
        "A_signed": sp.N(dM/2),
    }

# Examples
print("\nExample (non-collinear): P1=(0,0), P2=(1,0), P3=(0,1)")
ex1 = solve_three_point_circle((0,0), (1,0), (0,1))
for k,v in ex1.items():
    print(f"{k}: {v}")

print("\nExample (collinear): P1=(0,0), P2=(1,1), P3=(2,2)")
ex2 = solve_three_point_circle((0,0), (1,1), (2,2))
for k,v in ex2.items():
    print(f"{k}: {v}")

```

Exercise 10 - Quick classification drills

Question

For each implicit curve below, classify it and, when it is a circle, extract the center and radius by completing the square.
(a) $x^2 + y^2 - 6x + 8y + 9 = 0$.
(b) $2x^2 + 2y^2 + 4x - 12y + 1 = 0$.
(c) $x^2 + y^2 + 4x + 6y + 20 = 0$.
(d) $x^2 + 2xy + y^2 = 16$.

Explain your classification in one or two sentences per item, explicitly citing which conditions from Section 2 you used.

(a) $x^2 + y^2 - 6x + 8y + 9 = 0$

Complete the squares:

$$= (x-3)2 - 9, y2 + 8y = (y+4)2 - 16. \begin{aligned} x^2 - 6x &= (x-3)^2 - 9, \\ y^2 + 8y &= (y+4)^2 - 16. \end{aligned}$$

Plug back:

$$(x-3)^2 - 9 + (y+4)^2 - 16 + 9 = 0 \implies (x-3)^2 + (y+4)^2 - 16 = 0,$$

hence

$$(x-3)^2 + (y+4)^2 = 16.$$

Classification. $a = c = 1,\ b = 0$ (isotropic quadratic part), and the completed-square right-hand side is $R^2 = 16 > 0$. This is a circle with center $(3, -4)$ and radius $R = 4$.

(b) $2x^2 + 2y^2 + 4x - 12y + 1 = 0$

Group the quadratic terms and complete the squares inside:

$$2\big(x^2 + 2x\big) + 2\big(y^2 - 6y\big) + 1 = 0.$$

Now

$$x^2 + 2x = (x+1)^2 - 1, \qquad y^2 - 6y = (y-3)^2 - 9,$$

so

$$2\big[(x+1)^2 - 1 + (y-3)^2 - 9\big] + 1 = 0 \implies 2\big[(x+1)^2 + (y-3)^2 - 10\big] + 1 = 0.$$

Distribute:

$$2(x+1)^2 + 2(y-3)^2 - 20 + 1 = 0 \implies 2(x+1)^2 + 2(y-3)^2 = 19.$$

Divide by 2:

$$(x+1)^2 + (y-3)^2 = \frac{19}{2}.$$

Classification. $a = c = 2,\ b = 0$ (isotropic), and $R^2 = 19/2 > 0$. Circle with center $(-1, 3)$ and radius $R = \sqrt{19/2}$.

(c) $x^2 + y^2 + 4x + 6y + 20 = 0$

Complete the squares:

$$x^2 + 4x = (x+2)^2 - 4, \qquad y^2 + 6y = (y+3)^2 - 9.$$

Substitute:

$$(x+2)^2 - 4 + (y+3)^2 - 9 + 20 = 0 \implies (x+2)^2 + (y+3)^2 + 7 = 0,$$

i.e.

$$(x+2)^2 + (y+3)^2 = -7.$$

Classification. Quadratic part is isotropic ($a = c = 1,\ b = 0$), but the completed-square right-hand side is negative: $R^2 = -7 < 0$. By the degeneracy test from Exercise 6/Section 2, there are **no real solutions** (an "imaginary circle," i.e., empty locus).

(Equivalently, in the normalized parameters $2m = 4$, $2n = 6$, $q = 20$, we have $m^2 + n^2 - q = 4 + 9 - 20 = -7 < 0$.)

(d) $x^2 + 2xy + y^2 = 16$

Factor the quadratic form:

$$x^2 + 2xy + y^2 = (x+y)^2,$$

so

$$(x+y)^2 = 16 \implies x + y = 4 \ \text{ or } \ x + y = -4.$$

Classification. Here $a = c = 1$ but $b = 1 \neq 0$: the quadratic part is **not** isotropic (fails $b = 0$), so it cannot be a circle. The equation factors into a **pair of parallel lines** $x + y = \pm 4$ (a degenerate conic). This is consistent with the "mixed-term present" condition in Section 2: after the 45° rotation that diagonalizes A, one eigenvalue vanishes (rank-1 form), yielding two lines rather than a closed curve.

```python
# Exercise 10 – SymPy program: classify each implicit curve; for circles, extract center & radius
# Plain printed output (no LaTeX/HTML).

import sympy as sp

x, y = sp.symbols('x y', real=True)

# Define the four equations' left-hand sides
exprs = {
    "(a)": x**2 + y**2 - 6*x + 8*y + 9,
    "(b)": 2*x**2 + 2*y**2 + 4*x - 12*y + 1,
    "(c)": x**2 + y**2 + 4*x + 6*y + 20,
    "(d)": x**2 + 2*x*y + y**2 - 16,
}

def circle_params_from_expr(expr):
    """
    Try to match expr to x^2 + y^2 + 2 m x + 2 n y + q.
    Return (is_isotropic, m, n, q) if matchable; else (False, None, None, None).
    """
    # Collect coefficients
    poly = sp.Poly(expr, x, y)
    a = poly.coeffs()[poly.monoms().index((2,0))] if (2,0) in poly.monoms() else 0
    c = poly.coeffs()[poly.monoms().index((0,2))] if (0,2) in poly.monoms() else 0
    b = poly.coeffs()[poly.monoms().index((1,1))] if (1,1) in poly.monoms() else 0
    dx = poly.coeffs()[poly.monoms().index((1,0))] if (1,0) in poly.monoms() else 0
    ey = poly.coeffs()[poly.monoms().index((0,1))] if (0,1) in poly.monoms() else 0
    f  = poly.coeffs()[poly.monoms().index((0,0))] if (0,0) in poly.monoms() else 0

    # Isotropic quadratic part?
```

```
    isotropic = (b == 0) and (a == c) and (a != 0)

    if isotropic:
        # normalize by 'a' to match x^2 + y^2 + 2mx + 2ny + q
        m = sp.Rational(dx, 2*a)
        n = sp.Rational(ey, 2*a)
        q = sp.Rational(f, a)
        return True, sp.simplify(m), sp.simplify(n),
sp.simplify(q)
    else:
        return False, None, None, None

def classify(expr):
    # Try circle form
    is_circle_form, m, n, q = circle_params_from_expr(expr)
    if is_circle_form:
        S = sp.simplify(m**2 + n**2 - q)  # R^2
        if sp.simplify(S) > 0:
            return {
                "type": "circle",
                "center": (-m, -n),
                "R2": S,
                "radius": sp.sqrt(S),
                "degenerate": False
            }
        elif sp.simplify(S) == 0:
            return {
                "type": "point circle (degenerate)",
                "center": (-m, -n),
                "R2": S,
                "radius": 0,
                "degenerate": True
            }
        else:
            return {
                "type": "empty locus (imaginary circle)",
                "center": (-m, -n),
```

```
                "R2": S,
                "radius": sp.I*sp.sqrt(-S),  # purely formal
                "degenerate": True
            }
    else:
        # Not a circle (mixed term or anisotropic). Try factoring to see structure.
        fact = sp.factor(expr)
        return {
            "type": "not a circle (anisotropic or mixed term present)",
            "factorization": fact
        }

# Run classifications
for label, e in exprs.items():
    result = classify(e)
    print(f"{label}  {sp.simplify(e)} = 0")
    if result["type"] == "circle":
        print("  Classification: circle")
        print("  Center:", sp.simplify(result["center"]))
        print("  R^2:", sp.simplify(result["R2"]))
        print("  Radius:", sp.simplify(result["radius"]))
    elif result["type"] == "point circle (degenerate)":
        print("  Classification: point circle (degenerate)")
        print("  Center:", sp.simplify(result["center"]))
        print("  R^2:", sp.simplify(result["R2"]))
        print("  Radius:", 0)
    elif result["type"] == "empty locus (imaginary circle)":
        print("  Classification: empty locus (imaginary circle)")
        print("  Center:", sp.simplify(result["center"]))
        print("  R^2:", sp.simplify(result["R2"]))
    else:
        print("  Classification:", result["type"])
        print("  Factorization:", result["factorization"])
    print()

```